

基于哈夫曼编码的文件压缩与解压工具实验报告

2026 年 5 月 22 日

班级：2024217802 姓名：严子竣 学号：2024212622

1 需求分析

在计算机系统中，无损压缩技术广泛用于减少文件存储空间和传输带宽。哈夫曼编码作为一种经典的基于字符出现频率的最优前缀编码，被应用于 ZIP、PNG、JPEG 等众多文件格式的压缩环节。本实验要求基于哈夫曼编码实现一个通用的文件压缩与解压工具，支持对任意格式的文件进行压缩，生成真正的二进制压缩文件，并能够将压缩文件完全还原为原始文件。

1.1 程序功能

- 压缩功能：
 1. 以二进制方式读取源文件，统计所有字节值（0-255）的出现频率。
 2. 根据频率构建哈夫曼树，生成每个字节值的变长编码。
 3. 将原始文件名、原始文件大小、频率表等元数据与编码后的比特流一并写入压缩文件，形成可自解压的二进制存档。
- 解压功能：
 1. 从压缩文件中提取魔数、文件名、原文件大小、频率表。
 2. 利用频率表重建与压缩时完全一致的哈夫曼树。
 3. 按位读取压缩数据，沿哈夫曼树解码并逐字节写回目标文件，还原出完整原始文件。
- 错误处理：对文件打开失败、损坏的压缩文件、内存不足等情况均能输出清晰的错误提示并安全退出。

1.2 输入形式与值的范围

- 压缩阶段：
 - 通过命令行传入源文件路径和压缩文件路径，或在程序内部交互式提示输入。
 - 源文件可为任意格式、任意大小（受限于操作系统文件系统和内存）。
 - 压缩文件名可自定义，推荐以 `.huff` 为后缀。
- 解压阶段：

- 通过命令行传入压缩文件路径和输出目录（可选），或交互式输入。
- 压缩文件必须符合程序定义的格式，否则提示格式错误。
- 解压输出的目录若不存在，程序会报错并退出。

1.3 输出形式

- 压缩成功时输出：压缩完成：源文件 -> 压缩文件
- 解压成功时输出：解压完成：压缩文件 -> 输出文件
- 发生错误时输出具体原因，例如：无法打开输入文件 **xxx: Permission denied** 或 文件格式错误，不是有效的 **Huffman** 压缩文件。

1.4 测试数据

- 正确输入：一个普通文本文件、一张小图片或一个二进制文件。
- 边界输入：空文件（应提示无法压缩）、仅含单一字节的文件（压缩后应能正确还原）。
- 错误输入：不存在的源文件、无写入权限的输出路径、损坏的压缩文件（缺少魔数、频率表等）。

2 概要设计

2.1 问题解决思路

程序分为压缩和解压两个独立的可执行文件，各自采用以下策略：

1. 压缩流程：

- 分块读取源文件，用大小为 256 的桶统计各字节出现次数，得到频率数组。
- 利用最小堆构建哈夫曼树：为每个出现过的字节创建叶子节点，按频率进行堆排序；每次取出两个最小频率节点合并为新父节点，直到只剩一个根节点。
- 递归遍历哈夫曼树，为每个叶子生成二进制编码（向左为 0，向右为 1），编码位数最多 255 位，存储为定长字节数组。
- 写入压缩文件头：4 字节魔数 "HUFF"、2 字节原文件名长度、原文件名字符串、8 字节原始文件大小、256×8 字节频率表。
- 重新读取源文件，根据编码表将每个字节转换为变长比特流，通过位缓冲逐字节写入压缩文件。

2. 解压流程：

- 读取并验证魔数，确认文件格式。
- 读取文件名、原始大小、频率表。
- 根据频率表再次利用最小堆重建哈夫曼树，保证与压缩时完全一致。
- 按位读取压缩数据，从根节点开始，每读取一个比特决定向左或向右移动，到达叶子节点时输出该字节，然后回到根节点继续，直到还原全部原始大小字节。

2.2 数据结构定义

- 哈夫曼树节点:

```

1 typedef struct Node {
2     int symbol;           // 字节值0~255, 内部节点为-1
3     uint64_t freq;
4     struct Node *left, *right;
5 } Node;

```

- 最小堆:

```

1 typedef struct {
2     Node **array;
3     int size, capacity;
4 } MinHeap;

```

解压器中, 最小堆元素为结构体对 (Node*, freq), 定义形式稍异, 但逻辑等价。

- 编码表项:

```

1 typedef struct {
2     unsigned char code[CODE_BYTES]; // 存储二进制编码, 每字节存8位
3     int len;                         // 有效位数
4 } CodeTable;

```

- 位写入器 (压缩用):

```

1 typedef struct {
2     FILE *fp;
3     unsigned char buf;
4     int pos;           // 当前缓冲位0~7
5 } BitWriter;

```

- 位读取器 (解压用):

```

1 typedef struct {
2     FILE *fp;
3     unsigned char buf;
4     int pos;           // 当前剩余可读位数
5 } BitReader;

```

2.3 主程序流程与模块层次

2.3.1 压缩程序 compress.c

main

```

├─ 获取输入/输出文件名 (命令行或交互)
├─ 统计频率 (fread 循环)
├─ build_huffman_tree
│   └─ create_min_heap

```

```

|   |— insert_min_heap
|   |— extract_min / heapify
|— generate_codes (递归遍历树)
|— 写入文件头 (fwrite)
|— 编码并写入比特流
|   |— bit_writer_create
|   |— bit_writer_write (使用 code_table)
|   |— bit_writer_flush
|— 释放所有资源

```

2.3.2 解压程序 decompress.c

main

```

|— 获取输入文件/输出目录
|— 验证魔数, 读取文件头
|— 读取频率表
|— rebuild_huffman_tree
|   |— create_min_heap (变体)
|   |— extract_min / heapify (变体)
|— 构造输出路径并打开文件
|— 解码比特流
|   |— bit_reader_create
|   |— bit_reader_read (逐位返回)
|   |— 树遍历与字节输出
|— 释放所有资源

```

两个程序各自独立编译, 通过相同的频率表约定保证哈夫曼树的重建一致性。

3 详细设计

3.1 核心算法伪代码

3.1.1 哈夫曼树构建 (压缩与解压共用思路)

```

1 function build_huffman_tree(freq[256]) -> Node*:
2     heap = create_min_heap(256)
3     for symbol = 0 to 255:
4         if freq[symbol] > 0:
5             node = new Node(symbol, freq[symbol])
6             insert_min_heap(heap, node)
7     if heap.size == 0: return NULL
8     while heap.size > 1:
9         left = extract_min(heap)
10        right = extract_min(heap)
11        parent = new Node(-1, left.freq + right.freq)
12        parent.left = left; parent.right = right

```

```

13         insert_min_heap(heap, parent)
14     root = extract_min(heap)
15     销毁 heap->array 和 heap
16     return root

```

3.1.2 编码表生成（递归）

```

1 function generate_codes(root, arr[], depth, code_table[]):
2     if root.left != NULL:
3         arr[depth] = 0
4         generate_codes(root.left, arr, depth+1, code_table)
5     if root.right != NULL:
6         arr[depth] = 1
7         generate_codes(root.right, arr, depth+1, code_table)
8     if root.left == NULL and root.right == NULL: // 叶子
9         code_table[root.symbol].len = depth
10        将 arr[0..depth-1] 按位复制到 code_table[root.symbol].code 中

```

3.1.3 位写入

```

1 function bit_writer_write(bw, code_table[], symbol):
2     code = code_table[symbol].code
3     len = code_table[symbol].len
4     for i = 0 to len-1:
5         bit = (code[i/8] >> (7 - i%8)) & 1
6         if bit == 1: bw.buf |= (1 << (7 - bw.pos))
7         bw.pos++
8         if bw.pos == 8:
9             fwrite(&bw.buf, 1, 1, bw.fp)
10            bw.buf = 0; bw.pos = 0
11     end

```

3.1.4 解压主循环

```

1 while bytes_written < original_size:
2     bit = bit_reader_read(br) // 返回0/1或-1
3     if bit < 0: error
4     current = (bit == 0) ? current->left : current->right
5     if current->left == NULL and current->right == NULL:
6         fputc(current->symbol, fout)
7         bytes_written++
8         current = root

```

3.2 函数调用关系图（压缩）

```

main
├─ build_huffman_tree

```

```

|   |— create_min_heap / free_min_heap
|   |— insert_min_heap
|   |   |— swap_nodes
|   |— extract_min
|   |   |— heapify
|   |— free_min_heap
|— generate_codes
|— 文件写入操作 (fwrite / rewind)
|— bit_writer_create
|— bit_writer_write
|   |— fwrite
|— bit_writer_flush
|— free_tree

```

4 调试分析报告

4.1 调试过程中遇到的问题及解决方法

1. 问题 1: Permission denied 错误

运行时输出 无法创建压缩文件 xxx: Permission denied, 原因多为输出路径指向了只读目录或系统保护目录。

解决方法: 增加详细的错误输出 (使用 `strerror(errno)`), 并在程序启动时允许交互式输入文件名, 默认在当前目录生成输出文件, 避免权限问题。

2. 问题 2: 空文件处理

若对空文件直接压缩, 频率表全零, 建树函数返回 `NULL`, 后续编码读写会崩溃。

解决方法: 在统计完总字节数后检查 `total_bytes == 0`, 立即输出提示并退出, 不进入压缩流程。

3. 问题 3: 单一字节文件编码长度为 0

当文件只有一种字节时, 该字节的哈夫曼编码可能被错误地设为 0 位, 导致压缩文件不含有效编码, 解压失败。

解决方法: 在建树逻辑中, 若堆中仅一个节点, 直接将其作为根节点返回; 此时 `generate_codes` 对该节点生成的编码长度为 1 (根据需要设定), 保证编码有效。经测试, 现有实现已正确处理此种情况。

4.2 算法时空复杂度分析

• 压缩阶段:

- 统计频率: 一次遍历源文件, 时间复杂度 $O(N)$, N 为文件字节数。
- 构建哈夫曼树: 符号最多 256 个, 堆操作复杂度 $O(256 \log 256)$, 常数量级。
- 编码生成: 递归遍历全树, 同样常数时间。
- 编码写入: 每个字节根据编码长度进行位写入, 整体复杂度 $O(N)$ 。

- 空间复杂度：频率表、编码表、树节点等均占用常量级内存，位缓冲仅 1 字节，故空间开销为 $O(1)$ 额外内存。

- 解压阶段：

- 读取头信息：常数时间。
- 重建哈夫曼树：与压缩相同，常数时间。
- 解码比特流：每处理一位沿树走一步，总位移次数等于压缩后的比特数，最坏情况下等于 $O(N)$ （若编码长度远小于 8 位，可能略多，但总量级仍为 $O(N)$ ）。
- 空间复杂度：同样为 $O(1)$ 额外内存（除原始文件数据外）。

4.3 改进设想

- 可以将多个文件打包成一个 `.huff` 存档。
- 在压缩、解压时增加进度显示，提升用户体验。

5 用户使用说明

直接运行对应可执行文件，按提示输入文件名：

1. 压缩：

- (a) 双击 `compress.exe` 或在命令行运行。
- (b) 输入要压缩的源文件名，例如 `photo.jpg`。
- (c) 输入压缩后的文件名，例如 `photo.huff`（可仅指定文件名，默认生成在当前目录）。
- (d) 程序将显示“压缩完成：photo.jpg -> photo.huff”。

2. 解压：

- (a) 双击 `decompress.exe` 或在命令行运行。
- (b) 输入要解压的压缩文件名，例如 `photo.huff`。
- (c) 输入解压输出目录（直接回车则解压到当前目录）。
- (d) 程序将显示“解压完成：photo.huff -> photo.jpg”。

例如：

```
compress.exe test.png test.huff
decompress.exe test.huff .\output
```

6 测试结果

6.1 测试环境

- 操作系统：Windows 11
- 编译器：MinGW-W64 GCC 8.1.0
- 编译选项：-std=c11 -Wall（可选关警告）

6.2 测试用例 1：标准文本文件压缩解压

测试步骤：

1. 创建文本文件 `test.txt`，内容为“hello world this is a test file for huffman coding”。
2. 执行 `compress.exe test.txt out.huff`。
3. 执行 `decompress.exe out.huff`，得到还原文件 `test.txt`。
4. 对比原文件和还原文件的 MD5 值。

结果：压缩成功，解压后的文件与原文件完全一致，MD5 码相同。压缩文件大小比原文件大幅减小（本例中约减少 30%）。

6.3 测试用例 2：二进制图片文件

测试数据：一张 128×128 的 PNG 图片，大小 12,345 字节。

操作：压缩为 `img.huff`，再解压。

结果：复原图片与原图片逐字节相同，可正常打开查看。

6.4 测试用例 3：单一字节文件

测试数据：文件 `single.bin` 包含 10,000 个字节 0x41。

操作：压缩后大小为几百字节（主要存储头信息和频率表），解压得到完全一致的 10,000 个 0x41。

结果：极限情况处理正确。

6.5 测试用例 4：空文件

测试数据：一个 0 字节的空文件 `empty.dat`。

操作：运行压缩程序，程序立即输出“输入文件为空，无法压缩。”并返回错误码。

结果：正确处理边界情况。

6.6 测试用例 5：损坏的压缩文件

测试数据：创建一个包含非“HUFF”魔数的文件。

操作：运行解压程序，输出“文件格式错误，不是有效的 Huffman 压缩文件。”

结果：错误检测有效。

6.7 测试用例 6：权限不足的输出路径

测试数据：试图将压缩文件写入 `C:\Windows\System32\test.huff`。

操作：运行压缩程序，系统返回“无法创建压缩文件... Permission denied”。

结果：出现错误提示，程序正常退出。